

Mini-projet 16 — Scanner en parallèle (threads)

Module D · RÉSEAU · Notions : **concurrency** · **ThreadPoolExecutor** · **submit** / **as_completed** · **I/O-bound** · Durée : 2 h · Parcours Python — Ednir

Pourquoi ce mini-projet ?

Ton scanner teste les ports **un par un**. Chaque port filtré attend jusqu'au timeout. Fais le calcul : $1000 \text{ ports} \times 1 \text{ s} = \text{plus de 16 minutes}$ 🕒. Or 99% de ce temps, ton programme **attend** le réseau sans rien faire. La solution : lancer **plein de tests en même temps**. Pendant qu'un thread attend une réponse, les autres travaillent. Un scan de plusieurs minutes tombe à **quelques secondes**. C'est le mini-projet qui rend ton outil vraiment utilisable.

1. Pourquoi les threads marchent ici

Un scan de ports est **I/O-bound** : le goulot n'est pas le calcul (le CPU), c'est l'**attente** de la réponse réseau. Pendant cette attente, le thread ne fait rien.

L'image : un seul serveur qui prend 100 commandes l'une après l'autre, en attendant chaque plat cuire avant de passer à la table suivante. Avec des threads, tu prends les 100 commandes d'un coup et tu passes à la suivante pendant que ça cuit. Le temps total s'effondre.

Nuance (le GIL) : tu as peut-être entendu que Python ne fait « pas de vrai parallélisme » à cause du GIL. C'est vrai pour le **calcul pur** (CPU-bound). Mais pour l'**attente réseau** (I/O-bound, notre cas), le GIL est **relâché** pendant l'attente → les threads sont parfaitement efficaces. Pour un scanner, les threads sont le bon outil.

2. L'outil moderne : ThreadPoolExecutor

Plutôt que de manipuler des threads à la main, on utilise un **pool** (une équipe de threads) fourni par `concurrent.futures`. Tu lui donnes des tâches, il les répartit.

```
from concurrent.futures import ThreadPoolExecutor, as_completed

with ThreadPoolExecutor(max_workers=100) as executor:  # une équipe de 100 threads
    futur = executor.submit(etat_port, "127.0.0.1", 8080)
    # submit LANCE la tâche en arrière-plan et rend un "future" (une promesse)
    print(futur.result())  # .result() attend et récupère le retour de etat_port
```

- `ThreadPoolExecutor(max_workers=100)` = une équipe de 100 threads. Le `with` les ferme proprement à la fin (ton MP13 !).
- `executor.submit(fonction, arg1, arg2)` = « lance `fonction(arg1, arg2)` en arrière-plan », et renvoie un **future**.
- `futur.result()` = récupère le résultat (attend qu'il soit prêt si besoin).

3. Lancer 1000 tâches et récupérer les résultats

Le motif standard : soumettre une tâche par port, puis récupérer les résultats **au fur et à mesure** qu'ils arrivent avec `as_completed`.

```
def scan_parallele(ip, ports, timeout=1, max_workers=100):
    ouverts = []
    with ThreadPoolExecutor(max_workers=max_workers) as executor:
        # on soumet une tâche par port, et on garde le lien future -> port
        futurs = {executor.submit(etat_port, ip, p, timeout): p for p in ports}
        for futur in as_completed(futurs):  # dès qu'un résultat est prêt
            port = futurs[futur]  # on retrouve DE QUEL port il s'agit
            if futur.result() == "ouvert":
                ouverts.append(port)
    return sorted(ouverts)
```

Le dictionnaire `{future: port}` : c'est une compréhension de dict (MP08/MP12 !). Quand un future se termine, `as_completed` te le donne — mais dans le **désordre** (celui qui finit en premier sort en premier). Le dict te permet de retrouver à quel **port** ce future correspondait. D'où le `sorted()` à la fin pour réordonner.

4. Le gain, chiffré

```
import time

debut = time.time()
ouverts = scan_parallele("127.0.0.1", list(plage_ports(1, 1000)))
print(f"{len(ouverts)} ports ouverts en {time.time()-debut:.2f}s")
# Séquentiel (1 par 1) : plusieurs minutes sur des ports filtrés
# Parallèle (100 threads): quelques secondes
```

Ta mission

Objectif : un scanner parallèle qui utilise ton `etat_port` (MP15) et ton `plage_ports` (MP11).

- Reprends `etat_port` et `plage_ports` (recopie-les en haut du fichier).
- Écris `scan_parallele(ip, ports, timeout=1, max_workers=100)` qui :
 - crée un `ThreadPoolExecutor` ;
 - soumet une tâche `etat_port` par port ;
 - récupère les résultats et **renvoie la liste triée des ports ouverts**.

Tests (serveur `http.server 8080` lancé) :

```
ports = list(plage_ports(8000, 8100))
print(scan_parallele("127.0.0.1", ports))
# -> [8080] (le seul ouvert dans la plage)

# Le test qui montre la vitesse : chronomètre un vrai scan
import time
debut = time.time()
ouverts = scan_parallele("127.0.0.1", list(plage_ports(1, 1000)))
print(f"{ouverts} en {time.time()-debut:.2f}s")
```

Bonus (frappant) : écris une version **séquentielle** `scan_sequentiel(...)` (juste une boucle `for`), et compare les deux temps sur la même plage. Tu verras l'écart en direct.

✓ Critères de réussite

- `scan_parallele("127.0.0.1", list(plage_ports(8000, 8100)))` renvoie `[8080]`.
- La liste renvoyée est **triée** (le parallèle finit dans le désordre, d'où `sorted`).
- Tu utilises `ThreadPoolExecutor` dans un `with`.
- (Bonus) tu constates que le parallèle est nettement plus rapide.

Si tu bloques — indices progressifs

Étape	Indice
Import	<code>from concurrent.futures import ThreadPoolExecutor, as_completed</code> .
Soumettre les tâches	<code>futurs = {executor.submit(etat_port, ip, p, timeout): p for p in ports}</code> — un future par port.
Récupérer	<code>for futur in as_completed(futurs):</code> puis <code>port = futurs[futur]</code> et teste <code>futur.result() == "ouvert"</code> .
Trier	<code>return sorted(ouverts)</code> — car les résultats arrivent dans le désordre.

Où ranger : `mini-projets/16-threads-parallele/scan_parallele.py`.

Attention : ne monte pas `max_workers` à 50 000 — au-delà de quelques centaines, tu satures ta machine. 100-200 est un bon réglage pour un scan local.

Quand c'est prêt : `git add .` → `commit` → `push`, puis « c'est poussé » + le temps que tu obtiens.